INSTRUCTION ARCHITECTURES AND COMPILERS

NB: Questions 1–5 relate to the Instruction Architectures half of the module. Questions 6–10 relate to the Compilers half of the module. A summary of the RISC-V RV32I instruction set is provided at the back of this paper.

1. Use the instruction set summary provided, translate the following RISC-V RV32I assembly language instructions into machine code instruction as 32-bit hexadecimal numbers.

   a) `lw s0, 255(t1)` [3]

   b) `xor a0, t1, t2` [3]

   Answer:
   This question tests a student's understanding of how instructions are encoded.

   - *16'h0ff32403*
   - *16'h00734533*

2. What are the assembly language instructions of the following RISC-V RV32I machine code instructions?

   a) `16'h000302b3` [4]

   b) `16'h00a12523` [4]

   Answer:
   This question tests a student's understanding of how to decode instruction from the machine code to assembly instructions.

   - *add t0,t1,zero*
   - *sw a0,10(sp)*

Answers

3. Figure 3.1 shows the design of a finite state machine `fsm` in SystemVerilog. The state machine has one input signal `x` and one output signal `z`. The state machine is controlled by the clock signal `clk`.

a) How many states are there? [1]

b) How is the state machine initialised? [1]

c) Draw the state diagram of this state machine. [4]

d) Complete the timing diagram shown in Figure 3.2 including the state of `fsm` at clock cycles T0 to T9. You may assume that the `rst` signal is low throughout, the input signal `x` changes shortly before the rising edge of the clock signal, and that `fsm` is initially at state `A`. To help you show your answer, a duplicate copy of Figure 3.2 is provided at the end of this paper. You can tear it off, annotate the diagram and attach it to the answer book. [4]

```
module fsm (
    input  logic    clk,    // clock
    input  logic    rst,    // rest state machine
    input  logic    x,      // input signal x
    output logic    z       // output signal z
);

    // define states
    typedef enum {A, B, C, D} my_states;
    my_state current_state, next_state;

    // state registers
    always_ff @(posedge clk)
        if (rst)    current_state <= A;
        else        current_state <= next_state;

    // next state logic
    always_comb
        case (current_state)
            A: if (x) next_state = B;
               else next_state = A;
            B: if (x) next_state = C;
               else next_state = A;
            C: if (x) next_state = A;
               else next_state = D;
            C: if (x) next_state = A;
               else next_state = B;
        default: next_state = current_state;
        endcase

    // output logic
    always_comb
        case (current_state)
            A: z = 0;
            B: z = 0;
            C: z = 1;
            D: z = 0;
        default: Z = 0;
        endcase
endmodule
```

Figure 3.1

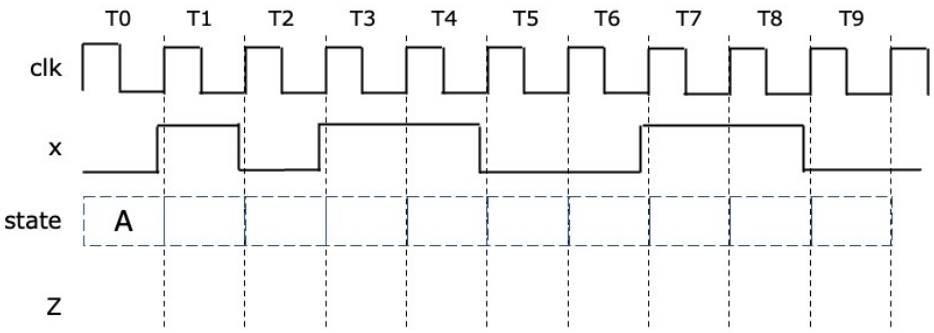


Figure 3.2

Answer:
This question tests a student's understanding of a finite state machine designed using SystemVerilog, the use of state diagrams and timing diagram to verify the hardware.

- *Four states: `A`, `B`, `C` and `D`.*
- *`fsm` is initialised when `rst` is asserted (high) to state `A` synchronously with the clock signal `clk`.*

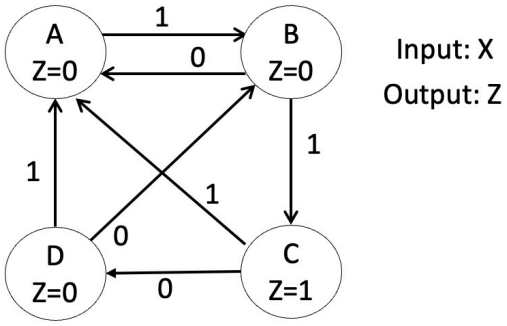


Figure 3.3

- *The state diagram is shown in Figure 3.3.*
- *The completed timing diagram is shown in Figure 3.4.*

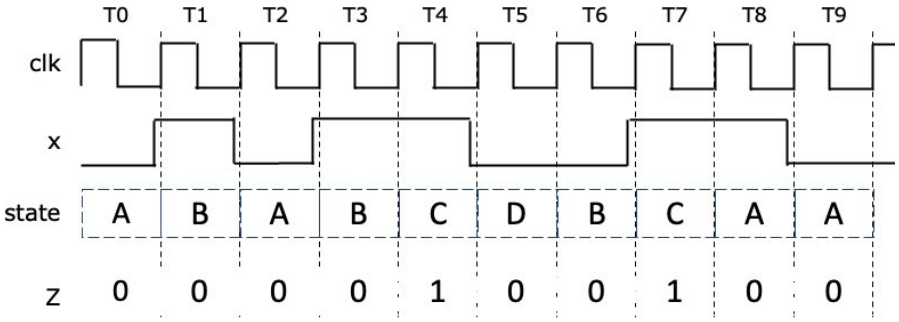


Figure 3.4

Answers

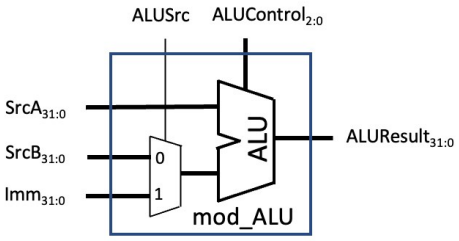
4. Figure 4.1 shows the module `mod_ALU` that implements the ALU hardware for the following five RISC-V RV32I instructions:

- `add rd, rs1, rs2`
- `sub rd, rs1, rs2`
- `sll rd, rs1, rs2`
- `addi rd, rs1, imm`
- `slti rd, rs1, imm`

The meanings of the `ALUControl[2:0]` signals for the four operations are also shown in Figure 4.1. You are responsible for the design of this `mod_ALU` module only.

a) Design the module `mod_ALU` (only this module) in SystemVerilog to execute these four instructions. [7]

b) Explain with the aid of a diagram how the bits in the 32-bit instruction can be used to produce `ALUSrc` and `ALUControl[2:0]` signals without any logic gates. [5]



$ALUControl_{2:0}$	Function
000	add
001	shift left logical
010	set less than
100	subtract

Figure 4.1

Answer:
This question tests a student's ability to map instructions to hardware. The question only uses R-type and I-type instructions. They will need to use the instruction set summary pages, and focus on the following instructions ONLY.

Register (R-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R-type	funct7							rs2					rs1					funct3			rd					opcode						

opcode	funct3	funct7	instruction	description	operation
7'h33	3'b000	7'h00	add rd, rs1, rs2	add	rd = rs1 + rs2
7'h33	3'b000	7'h20	sub rd, rs1, rs2	subtract	rd = rs1 - rs2
7'h33	3'b001	7'h00	sll rd, rs1, rs2	shift left logical	rd = rs1 << $rs2_{4:0}$

Immediate (I-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I-type	imm[11:0]												rs1					funct3			rd					opcode						

opcode	funct3	imm[11:5]	instruction	description	operation
7'h13	3'b000	-	`addi rd, rs1, imm`	add immediate	rd = rs1 + sign_ext (imm)
7'h13	3'b010	-	`slti rd, rs1, imm`	set less than immediate	rd = (rs1 < sign_ext(imm))

- *Here is the SystemVerilog code for `mod_ALU`.*

ACSWIS

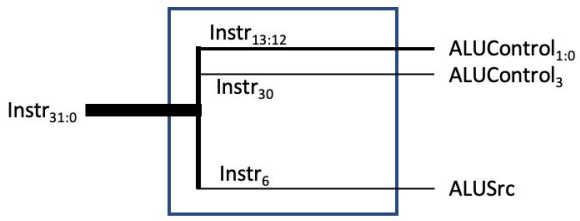
```
module ALU (
    input  logic [31:0] SrcA,       // rd1 from register file
    input  logic [31:0] SrcB,       // rd2 from register file
    input  logic [31:0] Imm,        // Immediate operand
    input  logic        ALUSrc,     // Select between rd2 or immediate operand
    input  logic [2:0]  ALUControl, // ALU control signals
    ouput  logic [31:0] ALUResult   // ALU output
);
    logic [31:0]    Op2;            // 2nd operand to ALU

    // Select source of op 2
    assign Op2 = (ALUSrc ? SrcB : Imm);

    // ALU
    always_comb
        case (ALUControl)
            3'b000: ALUResult = SrcA + Op2;         // add and addi
            3'b001: ALUResult = SrcA << Op2[4:0];   // sll
            3'b010: ALUResult = (SrcA < Op2);       // slti
            3'b100: ALUResult = SrcA - Op2;         // sub
        default: ALUResult = 32'h0;
        endcase

endmodule
```

- *The opcode[6:0] for R-type instruction is 7'h33 and for I-type instruction is 7'h13. opcdoe[6:0] is Instr[6:0]. Therefore, bit 6 of the instruction determines ALUSrc: 0 for immediate operand, and 1 for register rd2 operand.*
  add and addi instructions are the same operations (just different 2nd operand) and is defined by funct3[2:0] = 3'b000. While sll and slt instructions are convenient defined by funct3[2:0] = 3'b001 and 3'b010 respectively. Therefore ALUControl[2:0] maps direction to funct3[1:0].
  sub instruction shares the same funct3 value with add/addi instructions. However, it is distinguished from add with funct7[6:0] = 7'h20 instead of 7'h00. Therefore it is sub instruction if funct[6] = 1, otherwise it is and add instruction. Since sub is the only instruction in this question that has ALUControl[2] = 1'b1, we can use funct7[5] = ALUControl[2] = Instr[30].
  This is a simple solution if one has a clear mind - otherwise, it is somewhat challenging.



Answers

5. Figure 5.1 shows an implementation of the RISC-V RV32I processor using five stages of pipelining when executing four instructions.

a) Explain what each of the pipeline stages does. [4]

b) What are data hazards in a pipelined CPU architecture? Briefly describe two techniques that can be used to mitigate such hazards. [4]

c) For the four instructions shown in Figure 5.1, identify where and when data hazards occur. To help you show your answer, a duplicate copy of Figure 5.1 is provided at the end of this paper. You can tear it off, annotate the diagram and attach it to the answer book. [6]

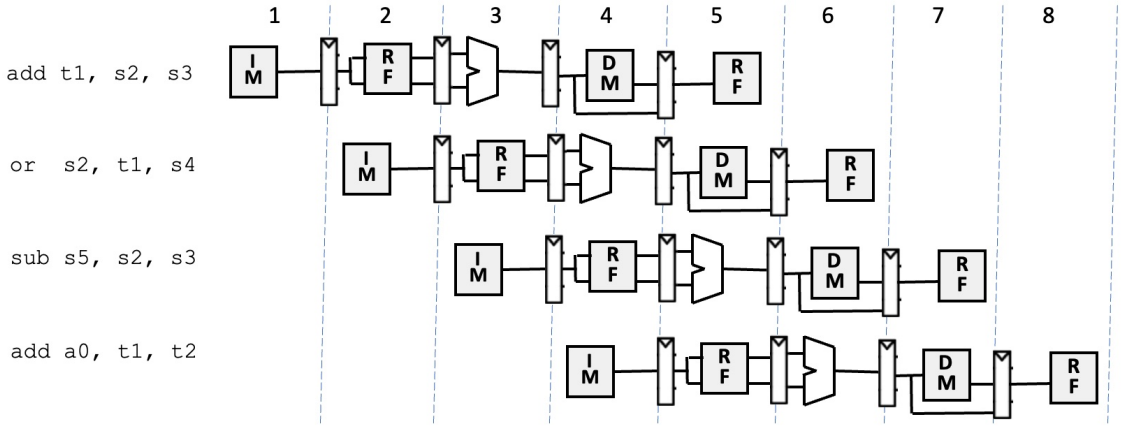


Figure 5.1

Answers

Answer:

(a)

i) Stage 1 IM - fetches instruction at memory location of PC and decode.

ii) Stage 2 RF - read operands from register file (for immediate instruction, construct immediate operand)

iii) Stage 3 ALU - perform ALU calculation

iv) Stage 4 DM - Data memory read/write

v) Stage 5 RF - Write back stage to register file

(b)

*Data hazard occurs due to **data dependency** in the code when an instruction requires a data that is in the process of being updated, but as yet unavailable. There are at three techniques that can be used to mitigate such hazard and only two are required:*

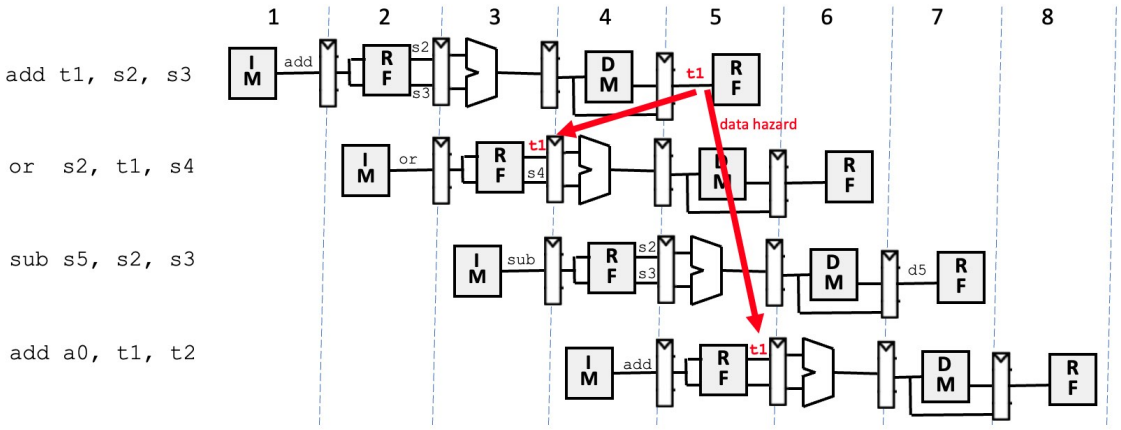
i) Work out the data dependency and add nop instruction as required to ensure that data has time to propagate to the stage requiring them.

ii) Stall a pipeline stage until require data is valid and available.

iii) Add bypass hardware in the pipeline processor to forward data to a later stage.

(c)

See the following diagram:



Answers

6. An application can be linked with a code library either statically (at compile time) or dynamically (at run time).

   a) Provide one reason why static linking could lead to better run-time performance than dynamic linking. [3]

   Answer:
   Here are two possible answers:

   - *Static linking avoids the extra layer of indirection for function calls that dynamic linking introduces.*
   - *Static linking can lead to better locality of reference because library functions are likely to be nearby in memory.*

   b) Provide one reason why dynamic linking could lead to better run-time performance than static linking. [3]

   Answer:
   Here are two possible answers:

   - *Widely shared library functions are unlikely to be paged out to disk by the OS.*
   - *The library may already be loaded in memory when application starts, which reduces both memory pressure and start-up overhead.*

7. A C compiler can support '`do...while`' loops by transforming them into `while` loops. Briefly describe this transformation, and explain why it may not be the most efficient compilation strategy. [6]

   Answer:
   It can transform '`do C while (B)`' into '`C; while (B) C`'. This is inefficient because the code for `C` is duplicated. Also, a `while` loop involves two jumps (a conditional one at the start of each iteration and an unconditional one at the end) but a `do...while` loop requires only one jump (a conditional jump at the end of each iteration). So, it may be more efficient to compile the `do...while` loop directly.

8. Consider the following C function.

```
int foo (int x) {
  return (x+1 < x);
}
```

   Explain how a compiler could optimise this function by exploiting the concept of 'undefined behaviour'. [6]

   Answer:
   The variable `x` is a signed integer, so is not allowed to overflow. If `x` is `INT_MAX` then the addition will overflow and the compiler can do anything. If `x` is not `INT_MAX` then the addition will not overflow and the compiler must return 0. So the function can be optimised to `int foo (int x) {return 0;}`.

9. Recall that (basic) regular expressions are constructed using the following grammar, in which c ranges over the symbols in the alphabet.

$$r ::= 0 \mid 1 \mid c \mid r + r \mid rr \mid r^* \mid (r)$$

   Throughout this question, let us use $\{\mathtt{a}, \mathtt{b}\}$ as our alphabet.

   a) Explain two ways in which a non-deterministic finite automaton (NFA) can exhibit non-determinism. [4]

ANSWERS

Answer:

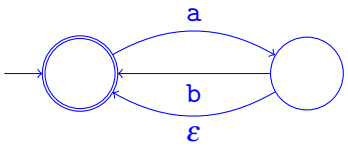
- *There may be two or more edges labelled with the same symbol coming out of the same state; any of those edges can be taken non-deterministically when that symbol appears.*
- *There may be ε-transitions, which can be taken or not taken (non-deterministically) at any point.*

b) Sketch an NFA that accepts the same language as the following regular expression.

$$(\mathtt{a}(1+\mathtt{b}))^*$$

[Hint: Your NFA should not require more than two states.] [6]

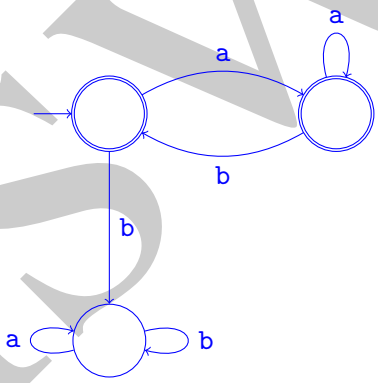
Answer:



Note that for full marks, the start states and the accepting states must be explicitly marked as shown.

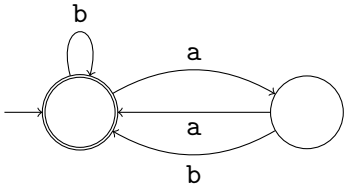
c) With the help of the Rabin–Scott subset construction or otherwise, sketch an equivalent *deterministic* finite automaton (DFA). [5]

Answer:



Note that for full marks, the start states and the accepting states must be explicitly marked as shown.

d) Write down a regular expression that accepts the same language as the following DFA. [4]



Answer:
$(\mathtt{b}+\mathtt{a}(\mathtt{b}+\mathtt{a}))^*$ *or* $(\mathtt{b}+\mathtt{ab}+\mathtt{aa})^*$ *or equivalent.*

Answers

e) Define the *length* of a regular expression as the number of (non-whitespace) characters it contains. For example, the length of the regular expression

$$(\mathtt{a}(1+\mathtt{b}))^*$$

is 9. If I have a regular expression of length N, is it the case that for every $M > N$ there exists a regular expression of length M that accepts the same language? Explain your answer. [5]

Answer:
Yes. You can concatenate the original regular expression with a sequence of $M-N$ 1s. For full marks, the answer should be fairly careful about where these 1s are placed (e.g. 'concatenated with the original regex'). Simply saying 'add some 1s' isn't quite precise enough, but gets the rough idea.

Answers

10. Consider the following type-inference rule for if-expressions.

$$\frac{\Gamma \vdash e_1 : \texttt{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau}$$

a) Explain why the rule *cannot* be used to infer a type for the following expression.

```
if false then 5 else true
```

[3]

Answer:
The rule requires the 'then' and 'else' branches to have the same type, but `5` *and* `true` *have types* `int` *and* `bool` *respectively.*

b) Propose a new rule that, if added to the existing set of rules, would allow a reasonable type to be inferred for the expression above. [5]

Answer:
With the following additional rule

$$\frac{\Gamma \vdash e_3 : \tau}{\Gamma \vdash \texttt{if false then}\ e_2\ \texttt{else}\ e_3 : \tau}$$

it would become possible to infer type `bool` *for the given expression.*

Answers

RISC-V Instruction Set Summary (1)

Register (R-type) Instructions

Bit position	31 30 29 28 27 26 25	24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
R-type	funct7	rs2	rs1	funct3	rd	opcode

opcode	funct3	funct7	instruction	description	operation
7'h33	3'b000	7'h00	add rd, rs1, rs2	add	rd = rs1 + rs2
7'h33	3'b000	7'h20	sub rd, rs1, rs2	subtract	rd = rs1 - rs2
7'h33	3'b001	7'h00	sll rd, rs1, rs2	shift left logical	rd = rs1 << $rs2_{4:0}$
7'h33	3'b010	7'h00	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
7'h33	3'b011	7'h00	sltu rd, rs1, rs2	set less than unsigned	rd = $(rs1 < rs2)^{unsigned}$
7'h33	3'b100	7'h00	xor rd, rs1, rs2	bit-wise XOR	rd = rs1 ^ rs2
7'h33	3'b101	7'h00	srl rd, rs1, rs2	shfit right logical	rd = rs1 >> $rs2_{4:0}$
7'h33	3'b101	7'h20	sra rd, rs1, rs2	shfit right arithmetic	rd = rs1 >>> $rs2_{4:0}$
7'h33	3'b110	7'h00	or rd, rs1, rs2	bit-wise OR	rd = rs1 | rs2
7'h33	3'b111	7'h00	and rd, rs1, rs2	bit-wise AND	rd = rs1 & rs2

Immediate (I-type) Instructions

Bit position	31 30 29 28 27 26 25 24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
I-type	imm[11:0]	rs1	funct3	rd	opcode

opcode	funct3	imm[11:5]	instruction	description	operation
7'h03	3'b000	-	`lb rd, imm(rs1)`	load byte immediate	rd = sign_ext ($[address]_{7:0}$)
7'h03	3'b001	-	`lh rd, imm(rs1)`	load half immediate	rd = sign_ext ($[address]_{15:0}$)
7'h03	3'b010	-	`lw rd, imm(rs1)`	load word immediate	rd = $[address]_{31:0}$
7'h03	3'b100	-	`lbu rd, imm(rs1)`	load byte unsigned	rd = zerp_ext ($[address]_{7:0}$)
7'h03	3'b101	-	`lhu rd, imm(rs1)`	load half unsigned	rd = zero_ext ($[address]_{15:0}$)
7'h13	3'b000	-	`addi rd, rs1, imm`	add immediate	rd = rs1 + sign_ext (imm)
7'h13	3'b001	7'h00	`slli rd, rs1, uimm`	Shift left logical immediate	rd = rs1 << uimm
7'h13	3'b010	-	`slti rd, rs1, imm`	set less than immediate	rd = (rs1 < sign_ext(imm))
7'h13	3'b011	-	`sltiu rd, rs1, uimm`	set less than imm. unsigned	rd = (rs1 < zero_ext(imm))
7'h13	3'b100	-	`xori rd, rs1, imm`	bit-wise XOR immediate	rd = rs1 ^ sign_ext (imm)
7'h13	3'b101	7'h00	`srli rd, rs1, uimm`	Shift right logical immediate	rd = rs1 >> uimm
7'h13	3'b101	7'h20	`srai rd, rs1, uimm`	Shift right arithmetic imm.	rd = rs1 >> uimm
7'h13	3'b110	-	`ori rd, rs1, imm`	bit-wise OR immediate	rd = rs1 | sign_ext (imm)
7'h13	3'b111	-	`andi rd, rs1, imm`	bit-wise AND immediate	rd = rs1 & sign_ext (imm)

Upper Immediate (U-type) Instructions

Bit position	31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
U-type	imm[31:12]	rd	opcode

opcode	instruction	description	operation
7'h17	`auipc rd, upimm`	add upper immediate to PC	rd = {upimm, 12'b0} + PC
7'h37	`lui rd, upimm`	load upper immediate	rd = {upimm, 12'b0}

RISC-V Instruction Set Summary (2)

Store (S-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S-type	imm[11:5]							rs2					rs1					funct3			imm[4:0]					opcode						

opcode	funct3	instruction	description	operation
7'h23	3'b000	`sb rs2, imm(rs1)`	store byte immediate	$[address]_{7:0} = rs2_{7:0}$
7'h23	3'b001	`sh rs2, imm(rs1)`	store half immediate	$[address]_{15:0} = rs2_{15:0}$
7'h23	3'b010	`sw rs2, imm(rs1)`	store word immediate	$[address]_{31:0} = rs2$

Branch (B-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B-type	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				[11]	opcode						

opcode	funct3	instruction	description	operation
7'h63	3'b000	`beq rs1, rs2, label`	branch if equal	If (rs1 == rs2) PC = BTA
7'h63	3'b001	`bne rs1, rs2, label`	branch if not equal	If (rs1 == rs2) PC = BTA
7'h63	3'b100	`blt rs1, rs2, label`	branch if less than	If (rs1 == rs2) PC = BTA
7'h63	3'b101	`bge rs1, rs2, label`	branch if greater than or equal	If (rs1 == rs2) PC = BTA
7'h63	3'b110	`bltu rs1, rs2, label`	branch if less than unsigned	If (rs1 == rs2) PC = BTA
7'h63	3'b111	`bgeu rs1, rs2, label`	branch if greater or equal unsigned	If (rs1 == rs2) PC = BTA

BTA = Branch Target Address, PC = Program Counter

Jump-and-link Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
J-type	[20]	imm[10:1]											[11]	imm[19:12]							rd					opcode						
jalr – I-type	imm[11:0]												rs1					3'b000			rd					opcode						

opcode	instruction	description	operation
7'h67	`jalr rd, rs1, imm`	jump and link register	PC = rs1 + sign_ext(imm), rd = PC + 4
7'h6f	`jal rd, label`	Jump and link	PC = JTA, rd = PC + 4

JTA = Jump Target Address, PC = Program Counter

Register names

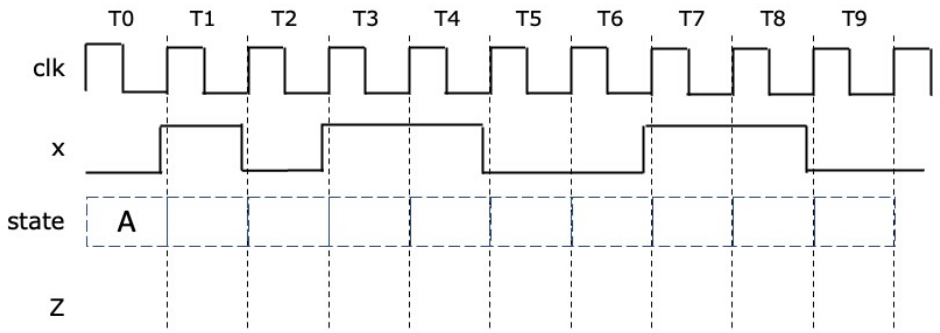
name	register number	usage
zero	x0	Constant value 0
ra	x1	Return Addrss
sp	x2	Stack Pointer
gp	x3	Global Pointer
tp	X4	Thread Pointer
t0 – t2	x5 – x7	Temporary registers
s0/fp	x8	Saved register/Frame Pointer
s1	x9	Saved register
a0 – a1	x10 – x11	Function arguments/ return values
a2 – a7	x12 – x17	Function arguments
s2 – s11	x18 – x27	Saved registers
t3 – t6	x28 – x31	Temporary registers

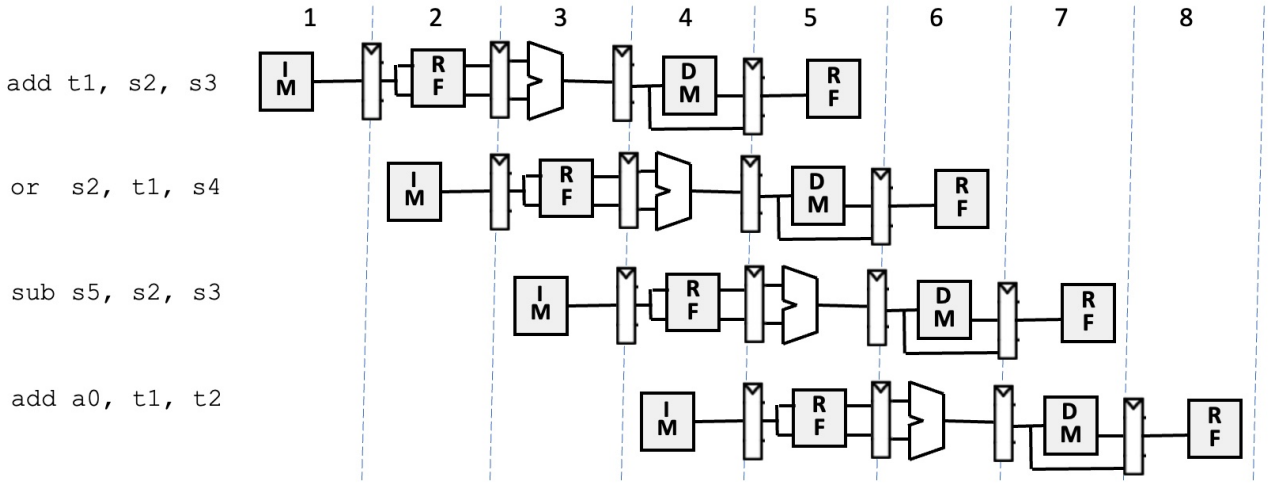
Abbreviations

imm	12-bit signed immediate ($imm_{11:0}$)
uimm	5-bit unsigned immediate ($imm_{4:0}$)
upimm	20-bit upper immediate ($imm_{31:12}$)
address	memory addres: rs1 + sign_ext($imm_{11:0}$)
[address]	data at memory location address
BTA	branch target address: PC + sign_ext({$imm_{12:1}$, 1'b0})
JTA	jump target address: PC + sign_ext({$imm_{20:1}$, 1'b0})
label	text label indicating instruction address
sign_ext	value sign-extended to 32 bits
zero_ext	value zero-extended to 32 bits

SUPPLEMENTARY SHEET FOR QUESTIONS 3 AND 5

You may tear off this page, annotate it with your answer, and attach it to the answer book.





Answers